

```

// מקבלת תור של מספרים (אפשר לשנות לטיפוס הרצוי)
// מחזירה כמות האיברים בתור, התור נשמאר
1 public static int len(Queue<Integer> q) {
2     Queue<Integer> tmp = new Queue<Integer>();
3     int count = 0;
4     while (!q.isEmpty()) {
5         count++;
6         tmp.insert(q.remove());
7     }
8     while (!tmp.isEmpty()) {
9         q.insert(tmp.remove());
10    }
11    return count;
12 }

```

```

// מקבלת תור של מספרים (אפשר לשנות לטיפוס הרצוי) ומיקום
// מחזירה איבר במיקום הנדרש, התור נשמאר
// מניחה שמיקום תקין ונמצא בתור
1 public static int at(Queue<Integer> q, int indx) {
2     Queue<Integer> tmp = new Queue<Integer>();
3     int x = 0;
4     while (!q.isEmpty()) {
5         if (indx == 0) {
6             x = q.head();
7         }
8         indx--;
9         tmp.insert(q.remove());
10    }
11    while (!tmp.isEmpty()) {
12        q.insert(tmp.remove());
13    }
14    return x;
15 }

```

הפעולה מחזירה true אם בתור q שהתקבל יש שני מספרים שכוכסם שווה לערך הפרמטר x. אחרת הפעולה מחזירה false.

דוגמה: עבור התור q שלמניכם ר-10= x הפעולה תחזיר true, כי יש בתור שני מספרים (1,9) שכוכסם שווה ל-10.

הניחוי שבתור q יש שני איברים לפחות.
אין צורך לשמור על התור q.
אורך להשתמש בשאלה זו במערך ובדינאמיקה מקושרת.

```

1 public static boolean twoSum(int[] a, int x) {
2     for (int i = 0; i < a.length; i++) {
3         for (int j = i+1; j < a.length; j++) {
4             if (a[i] + a[j] == x) {
5                 return true;
6             }
7         }
8     }
9     return false;
10 }
11 // Runtime complexity: O(n^2)
12
13 public static boolean twoSum(Queue q, int x) {
14     for (int i = 0; i < len(q); i++) {
15         for (int j = i+1; j < len(q); j++) {
16             if (at(q,i) + at(q,j) == x) {
17                 return true;
18             }
19         }
20     }
21     return false;
22 }
23 // Runtime complexity: O(n^3)

```

```

1 class NumCount {
2     private int num;
3     private int count;
4
5     public int getNum() { return this.num; }
6     public int getCount() { return this.count; }
7     public int setNum(int num) { this.num = num; }
8
9     public NumCount(int num) {
10         this.num = num;
11         this.count = 1;
12     }
13 }
14
15 // פונקציה עזר... מה מקבלת ומה מחזירה...
16 public static Node<NumCount> find(Node<NumCount> lst, int x) {
17     while (lst != null) {
18         if (lst.getValue().getNum() == x) {
19             return lst;
20         }
21         lst = lst.getNext();
22     }
23     return null;
24 }

```

```

1 class NumCount {
2     private int num;
3     private int count;
4
5     public int getNum() { return this.num; }
6 }
7
8 // פונקציה עזר... מה מקבלת ומה מחזירה...
9 public static Node<NumCount> insertOrdered(
10     Node<NumCount> lst,
11     Node<NumCount> node)
12 {
13     if (lst == null) {
14         return node;
15     }
16     int num = node.getValue().getNum();
17     if (num < lst.getValue().getNum())
18         node.setNext(lst);
19     return node;
20 }
21
22 Node cur = lst;
23 while (cur.hasNext() &&
24     cur.getNext().getValue().getNum() < num)
25 {
26     cur = cur.getNext();
27 }
28
29 node.setNext(cur.getNext());
30 cur.setNext(node);
31
32 return lst;
33 }

```

```

1 class OrderedList {
2     private Node<NumCount> lst;
3
4     public OrderedList() {
5         this.lst = null; // אומרים בפירוש שרשימה ריקה
6     }
7
8     public void insertNum(int x) {
9         Node<NumCount> node = find(this.lst, x);
10        if (node != null) {
11            int count = node.getValue().getCount();
12            node.getValue().setCount(count + 1);
13        } else {
14            node = new Node(new NumCount(x));
15            this.lst = insertOrdered(this.lst, node);
16        }
17    }
18 }

```

```

1 class OrderedList {
2     private Node<NumCount> lst;
3
4     public int valueN(int num) {
5         Node node = this.lst;
6         int indx = 0;
7         while (node != null) {
8             int count = node.getValue().getCount();
9             if (indx + count >= num) {
10                 return node.getValue().getNum();
11             }
12             indx += count;
13             node = node.getNext();
14         }
15         return node.getValue().getNum();
16     }
17 }

```

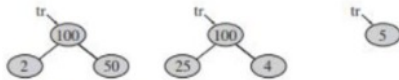
```

1 public static boolean isPrime(int num) {
2     for (int i = 2; i < num; i++) {
3         if (num % i == 0) {
4             return false;
5         }
6     }
7     return true;
8 }
9
10 public static int factorOf(int num) {
11     for (int i = 2; i < num; i++) {
12         if (num % i == 0) {
13             return i;
14         }
15     }
16     return num;
17 }

```

הפעולה מקבלת צומת ללא בנים (עלה) שערכו גדול מ-0. אם ערך הצומת הוא מספר ראשוני, הפעולה מחזירה false. אחרת, הפעולה מוסיפה לצומת שני בנים שערכי המכפלה שלהם שווה לערך הצומת, והערך של כל אחד מהם גדול מ-1. לאחר ההוספה הפעולה מחזירה true.

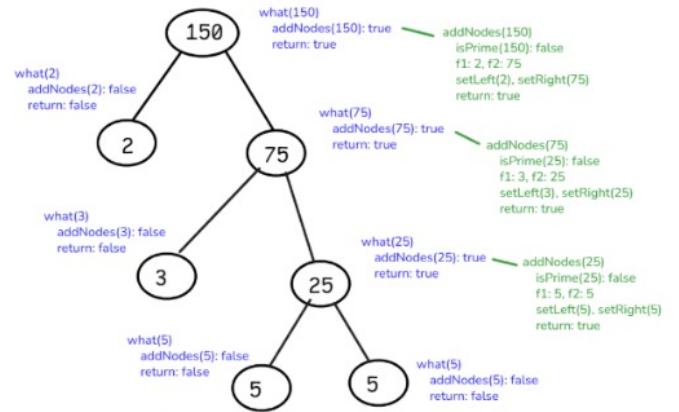
דוגמאות: בתום הפעולה הצמתים יכולים להיראות כך (עבור הערכים 5, 100, 100):



```

1 public static boolean addNodes(BinNode tr) {
2     int num = tr.getValue();
3     if (isPrime(num)) {
4         return false;
5     }
6     int f1 = factorOf(num);
7     int f2 = num / f1;
8     tr.setLeft(new BinNode(f1));
9     tr.setRight(new BinNode(f2));
10    return true;
11 }

```



```

public static void what (BinNode<Integer> tr)
{
    if (addNodes (tr))
    {
        what (tr.getLeft());
        what (tr.getRight());
    }
}

```